

# Why Machines Learn Notes

## Chapter 1.

### 1. Linear Relationship

Linear regression equation:

So let's say you have a table of values:

| x1 | x2 | y  |
|----|----|----|
| 4  | 2  | 8  |
| 1  | 2  | 5  |
| 0  | 5  | 10 |
| 2  | 1  | 4  |

There is a simple pattern in the data: In each row, the value of y is related to the corresponding values of x1 and x2.

The equation of the data turns out to be this:

$$y = x1 + 2x2$$

It can be rewritten as this:

$$y = w1x1 + w2x2$$

where  $w1 = 1$  and  $w2 = 2$

This is called a **linear relationship**, meaning that y depends only on the values of x1 and x2 (zero powers or products of x1 and x2).

The relationship between y, x1, and x2 is defined by the constants w1 and w2. These constants are called the coefficients, or **weights**, of the linear equation connecting y to x1 and x2.

If the equation gets too big, you need a better solution to calculate it. Consider this for a set of n weights, and using formal mathematical notation:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n = \sum_{i=1}^n w_ix_i$$

The expression on the right, using the sigma notation, is short-hand for summing all  $w_ix_i$ , where  $i$  takes on values from 1 to  $n$ .

It would be tiring to figure out the value of  $y$  by hand each time the values of  $x$  changed. That is where learning comes in. An algorithm of some kind learns the weights from previous data, then when new values of  $x$  are given, it can figure out  $y$  instantly as it already knows the weights.

**Linear regression models** learn how to predict an output by adjusting weights based on input features. The learning process uses previous data to calculate optimal weights for new predictions

**Ex)**

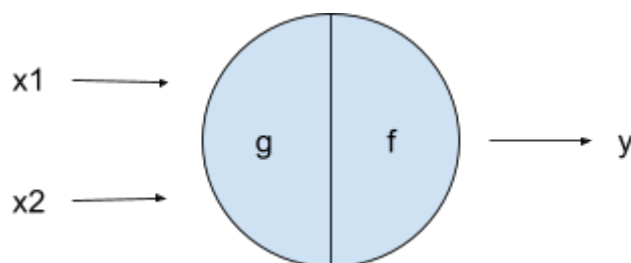
For example, if there was a linear relationship between the temperature, how cloudy the sky is, and how many people are at the beach:

$y$  represents the amount of people,  $x_1$  represents the temperature, and  $x_2$  represents how cloudy the sky is (in a numerical value between 0 and 1).

Using previous data, you could find the weights, or relationship, between these values. Then when given new data, such as the overcast and temperature of tomorrow, you could calculate how many people might be at the beach.

## 2. The First Artificial Neuron

The first advancement in AI was called the perceptron. However, let's start from the beginning. First in 1943, the neurode was invented. It was a computational model that one could use to implement certain basic Boolean logical operations such as AND, OR, NOT, and so on.



This is called the McCulloch-Pitts model where  $x_1$  and  $x_2$  can either be 0 or 1. In formal notation:

$$x_1, x_2 \in \{0, 1\}$$

This basically means that  $x_1$  and  $x_2$  are values in the set of  $\{0, 1\}$ , and they can only be either 0 or 1. The neurode's output  $y$  is calculated by first summing the inputs and then checking to see if that sum is greater than or equal to some threshold, *theta* ( $\theta$ ). If so,  $y$  equals 1; if not,  $y$  equals 0.

$$sum = x_1 + x_2$$

$$\downarrow$$

$$y = \begin{cases} 1 & \text{if } sum \geq \theta \\ 0 & \text{if } sum < \theta \end{cases}$$

This is just a piecewise function.

Generalizing this, the sum could be any amount of  $x$  values added up.

The model can be extended by increasing the number of inputs but by also letting some inputs be inhibitory, meaning  $x_1$  or  $x_2$  can be multiplied by -1. What this means in practice, is that if a value is **inhibitory**, no matter what the other values are, the output ( $y$ ) will always be 0. This can allow more complex logic.

A perceptron is an augmented McCulloch-Pitts neuron imbued with a learning algorithm.

